SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA1520 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management.

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	Dharshan A N	Register Number	192525370
Department / Section	BE AI&ML	Academic Year / Semester	2rd Year
Faculty Name	Dr.Arul Raj A.M	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Existing Data Centre and Requirements	3		Definition; characteristics; deployment models; scenario mapping
2	Virtualization Fundamentals and Hypervisors	3		Timeline; hardware evolution; relevance to university
3	Server Consolidation and Resource Calculations	3		Web/software evolution; SOA; APIs; scenario relevance
4	Virtual Machines and Containers	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Virtualized Storage and Network Infrastructure	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Resource Allocation, Scheduling and	5		Responsibility layers; five-application mapping; comparison; providers

	Monitoring			
7	Availability, Security and Disaster Recovery	3		Four or more variants; two justified choices; provider risks
8	Virtualized Cloud Data Centre Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Validation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Cost, Performance and Final Recommendation	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations		Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution		Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices		Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification		Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots		Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap		Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation		Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code		Flask REST API and Dockerfile provided in the proposed implementation
Virtualization		Docker container configuration documented; running-container evidence required
PaaS or SaaS		Managed web-service deployment procedure described; live deployment not verified
Functional test		Expected HTTP 200 response and JSON output documented; actual output required
Failure case		HTTP 404 test case documented; actual execution record required
Screenshots		Screenshot requirements identified; authentic screenshots need verification
Reproducibility		Build commands, run commands, endpoints and expected results specified

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	**Evidence expected**	**Status**	**Evaluator note**
1	Cloud definition, characteristics, deployment models and scenario mapping		All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance		Well explained with relevant examples
3	Internet software evolution, SOA and APIs		Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations		Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs		API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping		All five applications categorized with justification
7	XaaS variants and two justified		Six variants explained; DBaaS and

	selections		DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery		Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots		Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap		Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation		Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	**Status**	**Observation**
Source code		Python Flask REST API and Dockerfile documented
Virtualization		Docker build and run procedures provided; execution screenshot required
PaaS or SaaS		Cloud deployment steps described; actual deployment confirmation required
Functional test		HTTP 200 and expected JSON response documented; actual test output required
Failure case		HTTP 404 test case included; execution evidence required
Screenshots		Required screenshots identified but authenticity and completeness not verified
Reproducibility		Installation, configuration, commands and testing steps documented
Interpretation		Technical outcomes and implementation limitations explained
Code and report consistency		Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☐ Yes ☐ Partly ☐ No
☐ Existing data-centre requirements and limitations are identified.	☐Yes ☐ Partly ☐ No

☐ Virtualization and hypervisor types are explained.	☐Yes ☐ Partly ☐ No
☐ Server consolidation calculations include working and assumptions.	☐Yes ☐ Partly ☐ No
☐ VM and container comparison and workload mapping are included.	☐Yes ☐ Partly ☐ No
☐ Storage and network virtualization are addressed.	☐ Yes ☐Partly ☐ No
☐ Resource allocation and monitoring policies are included.	☐Yes ☐ Partly ☐ No
☐ Security risks, backup and disaster recovery are discussed.	☐Yes ☐ Partly ☐ No
☐ A labelled virtualized data-centre architecture diagram is included.	☐ Yes ☐Partly ☐ No
☐ Implementation screenshots and actual test evidence are provided.	☐ Yes ☐ Partly ☐ No
☐ Three-year cost analysis and migration roadmap are included.	☐ Yes ☐ Partly ☐ No

GENERAL INSTRUCTIONS

- Use clear headings and answer each task in the order given.
- Include diagrams, tables, calculations and screenshots wherever requested.
- State assumptions and units for all numerical calculations.
- Use credible technical references and cite sources consistently.
- Ensure that implementation evidence reflects work actually performed by the team.
- Submit the report as a well-formatted document with a title page, contents, references and appendices where required.

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

CSA1520 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title:

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management

(Total Marks – 50) Duration: 1 week

Scenario – University Data Centre Virtualization

A growing university operates an on-premises data centre that hosts its Student Information System, Learning Management System, Library Management System, Examination Portal, research applications and institutional website. Each major application runs on a separate physical server. Server resources are underutilized during normal periods, while admission, examination and result-publication periods create high demand. The university experiences rising electricity and maintenance costs, slow hardware procurement, limited scalability and inadequate disaster-recovery arrangements.

The current data-centre environment is assumed to have the following characteristics:

- 20 physical servers, each with 16 CPU cores, 64 GB RAM and 2 TB local storage.
- Average CPU utilisation is approximately 20%; peak utilisation reaches 90–95% during admission and examination periods.
- Most servers are dedicated to individual applications and cannot share unused capacity efficiently.
- There is no centralized virtual-machine (VM) management platform and no tested live-migration process.
- Backup is performed periodically, but a separate disaster-recovery site and documented recovery objectives are not available.
- The university wants to reduce infrastructure costs while improving availability, scalability, security and resource utilization.

You are appointed as a cloud infrastructure consultant. Analyse the existing environment and design a virtualized cloud data-centre solution. Your proposal must explain the role of hypervisors, virtual machines, containers, storage and networking virtualization, resource

allocation, monitoring, security and disaster recovery. Justify all major decisions using calculations, diagrams and practical evidence.

Design Challenge:

On Day 4, the university announces that it will introduce a new digital learning platform and extend selected services to 10 affiliated colleges. The expected peak workload will increase to five times the current peak. The university requires at least 99.9% service availability, stronger tenant isolation and controlled operating costs. Re-evaluate your original design and decide whether it should be retained, modified or redesigned..

TASK 1 – Analyse the Existing Data Centre and Identify Requirements (3 Marks)

Study the current data-centre environment and identify the reasons for adopting virtualization.

Explain the purpose of a data centre and identify its major components: compute, storage, networking, power, cooling and management.

Identify at least four limitations of the existing physical-server environment.

Define resource utilization, server consolidation, scalability, availability and fault tolerance.

Prepare a requirement table covering performance, capacity, security, availability and cost.

Evidence Expected: A concise assessment of the existing environment, clearly linking each identified problem to a virtualization or cloud-infrastructure requirement.

ANSWER

Assessment

A data centre combines compute, storage, networking, power, cooling, physical security and management to deliver application services. The university has 20 application-dedicated physical servers, yet average CPU use is only about 64/320 cores. Unused capacity is stranded, while 90–95% peak CPU at admission/exam periods causes pressure. Virtualization pools resources and makes capacity movable and measurable.

Requirement table

Performance	Meet peak admission/exam response-time targets; establish p95 latency SLO.
Capacity	Pool CPU/RAM/storage, retain N+1 capacity and a tested burst path.
Security	MFA/RBAC, segmentation, encryption, patching and audit for student data.
Availability	Cluster hosts, redundant network/storage/power, tested backup and DR.
Cost	Reduce idle hardware/power; track licenses, support and cloud use.

Key terms

Utilization is the share of provisioned capacity in use. Server consolidation places workloads from multiple underused servers on fewer, larger hosts. Scalability is the ability to increase/decrease capacity with demand. Availability is the proportion of time a service works as agreed. Fault tolerance continues service despite specified component failures.

Assessment in context

The 20-server estate has a total of 320 physical CPU cores, yet its estimated average demand is only 64 cores. This gap indicates that hardware is purchased and powered for a peak that occurs only during selected academic periods. Dedicated servers cannot share idle capacity, so one machine may be nearly idle while another application is close to saturation. Virtualization creates a resource pool from which VMs receive CPU, RAM, storage and network connectivity according to priority and measured demand. Consolidation can reduce the number of powered-on devices and simplify patching, inventory, monitoring and recovery. It also introduces shared failure domains, so redundancy and security controls must be designed at the cluster level.

Requirements and acceptance

Performance targets should describe real user journeys, such as p95 login latency or the time needed to submit an examination response. Capacity planning must include CPU, memory working sets, storage IOPS and network bandwidth; raw core count and terabytes do not show whether the database can meet response targets. Security requirements include least-privilege administration, protected student records, college tenant separation, encryption and audit retention. Availability requires redundant components and tested restoration, not just an uptime promise. A baseline collected over several weeks should measure CPU, RAM, disk latency/IOPS, network throughput and application response. Acceptance criteria can then require sustainable peak service, survival of one host failure, successful restoration from backup, and verified access controls.

Example workload requirements

During an examination window, the portal should prioritize authenticated logins, question delivery and answer submission above low-priority research batch jobs. The performance requirement can therefore include a measured p95 request latency and an error-rate ceiling for those operations. A capacity requirement should reserve sufficient database connections, storage IOPS and network headroom, not simply spare CPU. The security requirement includes protecting personally identifiable records and restricting administrative access to named staff. The availability requirement defines which services may be degraded and for how long. These examples show that requirements should be linked to service behavior, user impact and a measurable control.

Operational ownership

The infrastructure team owns host availability, virtual networking, storage protection and monitoring. Application owners own software support, data integrity, user acceptance and application-specific recovery steps. Security staff define identity, patch and audit controls, while procurement tracks refresh, support and licensing costs. Establishing these

responsibilities avoids an assumption that the hypervisor team can restore an application without the database owner or encryption keys. The requirement table should be reviewed with each owner before implementation and during annual capacity review.

Limitations and consequences

The current architecture also creates operational inconsistency. Different servers may have different operating-system versions, patch levels, monitoring agents and backup methods, increasing the chance that a security fix is delayed or a recovery procedure fails. Hardware failure can make a single application unavailable until a replacement is sourced and restored. Local disks on separate machines complicate shared recovery and limit workload movement. Procurement lead times make short-lived demand spikes difficult to address economically. Virtualization helps standardize images and automate provisioning, but it does not automatically fix outdated applications, poor data quality or weak account practices. Those issues need owners and separate remediation plans.

Capacity measures to collect

Record CPU utilization and ready time at five-minute intervals, active versus provisioned memory, disk queue and latency, network throughput and retransmissions, and application transactions per second. Include normal weeks and academic peaks. This baseline distinguishes a real CPU bottleneck from a storage or database issue and informs a defensible consolidation ratio. Keep the data long enough to compare semester cycles. Work with application owners to identify predictable calendar events, service dependencies and accepted degraded modes. The resulting requirement set becomes the basis for sizing, testing and cost review.

Additional design considerations

A useful service catalogue should identify the accountable owner, data classification, user groups, dependencies, business calendar, maintenance window and recovery priority for every application. This catalogue helps distinguish a high-availability requirement from a simple preference and prevents low-priority systems from consuming the same protection budget as examination services. Capacity and security reviews should be repeated after a major enrollment change or the onboarding of a new college.

TASK 2 – Explain Virtualization Fundamentals and Hypervisors (4 Marks)

Explain the technologies that enable multiple computing environments to share physical infrastructure.

Define virtualization and explain its purpose in a cloud data centre.

Explain the role and working of a hypervisor.

Differentiate Type-1 (bare-metal) and Type-2 (hosted) hypervisors with suitable examples such as VMware ESXi, Microsoft Hyper-V, KVM and Oracle VirtualBox.

Compare full virtualization, para-virtualization, hardware-assisted virtualization and OS-level virtualization (containers).

Recommend a suitable hypervisor for the university and justify the selection.

Evidence Expected: A labelled virtualization architecture diagram, comparison table and justified hypervisor recommendation.

ANSWER

Fundamentals

Virtualization abstracts CPU, memory, storage and networking into software-defined resources. A hypervisor maps virtual CPUs/devices to hardware, schedules execution, isolates guests and manages VM lifecycle. Intel VT-x/AMD-V and IOMMU support efficient and controlled guest/device execution.

Comparison

Type 1	Runs on hardware; ESXi, Hyper-V, KVM; preferred for production servers.
Type 2	Runs on a host OS; VirtualBox; useful for labs, with an added host layer.
Full virtualization	Presents virtual hardware so unmodified guest OSs run; broad compatibility.
Para-virtualization	Guest uses hypercalls/paravirtual drivers to reduce I/O overhead.
Hardware assisted	CPU virtualization extensions execute guests efficiently; standard modern foundation.
OS-level	Containers isolate processes using namespaces/cgroups while sharing a kernel; lightweight but shared-kernel boundary.

Architecture

Users → firewall/load balancer → virtual switch and segmented networks → VMs (guest OS + apps) or containers → Type-1 hypervisor → physical hosts, shared storage and redundant NICs. Recommend KVM or ESXi after comparing support, staff skills and licensing; Type-1 clustering with centralized management, hardware support, live migration and backup integration is required.

Hypervisor mechanics

A hypervisor maps guest virtual CPUs to physical processors, virtual memory to physical memory, and virtual storage/network devices to host resources. It schedules CPU execution, mediates access to devices and maintains isolation between guests. Hardware virtualization extensions support safe execution of guest operating systems; paravirtual drivers can improve I/O efficiency. The management plane supplies inventory, templates, role-based administration, monitoring, migration and policy enforcement. This plane is security-critical because an administrator who can alter images, virtual networks or snapshots can affect every hosted workload. Management access should use a dedicated network, strong authentication, least privilege and auditable change control.

Selection criteria

A Type-1 platform is the appropriate production baseline because it runs on server hardware and supports centralized cluster management. KVM, ESXi and Hyper-V are plausible

options; the final selection should compare hardware compatibility, staff skills, support response, backup integration, live migration, licensing and lifecycle cost. A lab proof-of-concept should verify VM creation, patching, workload relocation, restore and host-failure recovery. Avoid choosing solely on initial license price: training, management tooling and support can dominate the three-year cost. VM isolation is strong but not absolute; network segmentation, guest patching and protected management credentials remain essential.

Technology comparison in this scenario

Full virtualization is appropriate when the university must run unmodified operating systems. Hardware-assisted virtualization is common in production because current CPUs provide execution support; it does not remove the need for a properly configured hypervisor. Para-virtual drivers can improve disk and network behavior. Containers provide OS-level isolation and use fewer resources but share a kernel. These are complementary options: the campus can run a Type-1 hypervisor for the foundational cluster, then place Kubernetes worker nodes in VMs and deploy selected stateless applications as containers.

Management controls

Use approved VM templates with supported operating systems, security updates, monitoring agents and standard disk/network settings. Restrict who can create templates and who can alter virtual switches. Log administrator actions and review them independently. A hypervisor cluster should be patched in a rolling manner: evacuate a host, patch and validate it, then return workloads before moving to the next host. Maintain compatibility across the cluster so migrations remain possible. The operational proof should demonstrate that a host can be drained, serviced and returned without an unplanned outage.

Evolution and compatibility

Server virtualization grew from partitioning large mainframes to hypervisors that let standard x86 servers run many guest systems. Advances in CPU extensions, memory management, high-speed networking and shared storage made server consolidation practical. Web services and cloud platforms added APIs for provisioning resources on demand. For this university, the evolution means that infrastructure can be described through templates and policies rather than manual configuration on each physical server. It also means administrators need version control and automation to prevent configuration drift. Older applications may still depend on hardware or operating systems that need a VM boundary; modernization can proceed independently.

Types in a cloud environment

Full virtualization presents virtual hardware and is useful for unmodified guest systems. Para-virtual drivers improve communication for selected storage and network operations. Hardware-assisted virtualization uses processor features to execute guests with lower

overhead. OS-level virtualization isolates processes through kernel facilities and supports fast container deployment. Containers are not mini-VMs: the shared kernel means a host-kernel flaw may affect multiple workloads. A sensible arrangement can combine methods, using a secure Type-1 hypervisor at the hardware layer and containers only for services whose runtime and security profile have been reviewed.

Additional design considerations

Document the chosen platform's version, support period, hardware compatibility list, firmware requirements and cluster configuration. Keep management accounts separate from guest operating-system accounts, and use a controlled change process for host settings. This documentation makes maintenance repeatable and helps a second administrator recover the platform during an incident. Test the complete management workflow, including alerts and audit records, rather than testing only that a VM boots.

TASK 3 – Design Server Consolidation and Calculate Resource Requirements (5 Marks)

Propose a consolidation plan to reduce the number of physical servers while maintaining sufficient capacity.

Explain physical-to-virtual (P2V) migration and server consolidation.

Estimate the total CPU and memory demand using the stated server specifications and average utilization. State all assumptions.

Assume a target average CPU utilization of 65% on the consolidated hosts. Calculate the minimum number of hosts required based on CPU capacity, then check whether memory capacity changes the result.

Include a reasonable capacity reserve for peak demand and host failure; show the calculation clearly.

Discuss resource overcommitment, CPU scheduling, memory ballooning and the noisy-neighbour problem.

Evidence Expected: Worked calculations showing CPU and memory requirements, assumptions, capacity reserve and the recommended number of physical hosts. A final number without working will not receive full credit.

ANSWER

Worked sizing

Totals: 320 cores, 1,280 GB RAM, 40 TB raw local. Average CPU = 320×0.20=64 cores. Peak = 320×0.95=304 cores. Assume average memory follows 20% CPU only as a rough placeholder: 1,280×0.20=256 GB; measure actual memory.

Host calculation

With 16-core/64-GB hosts and a 65% target, usable CPU/host = 16×.65=10.4 cores. Average: ceil(64/10.4)=7; N+1 reserve gives 8 hosts. Current peak: ceil(304/10.4)=30; N+1 gives 31 hosts to retain the target after one failure. Memory at 65%=41.6 GB/host; ceil(256/41.6)=7 plus one=8, so CPU peak sizing determines the result. The initial 8-host cluster covers average load but not current peak at the target. Phase toward 31 hosts or validated burst capacity and admission control.

Operational controls

Avoid routine CPU overcommit on database/exam workloads; test modest overcommit for stateless web. Reservations protect critical VMs; limits constrain them; shares prioritize contention. CPU scheduling time-slices vCPUs. Ballooning reclaims guest memory but can hurt latency; do not rely on swapping. Monitor CPU ready/steal, memory pressure, storage latency and noisy-neighbour effects.

Interpreting the host calculation

Eight hosts represent the average-load footprint with one host reserved for failure. They do not support the stated peak while maintaining the 65% target. Thirty-one hosts allow one failure and leave thirty hosts: 30×16×0.65 = 312 core-equivalents, slightly above the estimated peak of 304. This narrow margin shows why the result is a planning estimate rather than a guarantee. A 32-host pool would provide more operating margin if the budget supports it. Hardware generations, clock rates, CPU-ready time and application licensing may alter the equivalent capacity; benchmark representative workloads before procurement.

Memory and storage validation

Memory use is not stated, so matching the 20% CPU figure to RAM produces only a rough average estimate. A database may use large caches while consuming little CPU; an analytics job may do the opposite. Measure active memory, working-set size, ballooning, swapping and peak simultaneous use. Reserve host memory for the hypervisor, management services and failover. Storage also needs IOPS, throughput and latency sizing. Forty terabytes of raw local disks is not 40 TB of usable protected capacity after redundancy, filesystem overhead, snapshots and backup copies. Track growth and backup-window duration. P2V migration should begin with an application inventory, dependency map, recoverable backup and rollback plan, followed by a low-risk pilot and owner validation.

Sizing sensitivity

If the true peak is 90% rather than the conservative 95%, demand is 320×0.90=288 core-equivalents. At 10.4 usable cores per host, ceil(288/10.4)=28 hosts carry that load, and N+1 requires 29. At 95%, the requirement is 31. This two-host difference illustrates why sizing should preserve the range and assumptions, not report only a single precise-looking number.

If an organization instead allows peak CPU to exceed 65% temporarily, fewer hosts could carry the load, but response-time and failover behavior must be measured. A temporary utilization excursion is a business decision that requires documented acceptance.

Practical P2V checks

Before conversion, identify application dependencies such as licensing dongles, hardware security modules, backup agents, scheduled jobs, static routes and external integrations. Confirm that the guest operating system is supported on the target hypervisor. Capture a pre-migration backup and record CPU, memory and response baselines. After migration, check boot, network identity, database consistency, application logs and scheduled tasks. Compare performance at the same workload. Keep the original server untouched until the owner signs off, then decommission according to the retention and secure-erasure policy. This reduces risk of a rollback that cannot be performed because the source was changed or erased too early.

Calculation summary table

Measure	Working	Result
Physical CPU	20 × 16 cores	320 cores
Average CPU demand	320 × 20%	64 core-equivalents
Peak CPU demand	320 × 95%	304 core-equivalents
Usable per target host	16 × 65%	10.4 core-equivalents
Average hosts plus N+1	ceil(64/10.4)+1	8 hosts
Peak hosts plus N+1	ceil(304/10.4)+1	31 hosts
Average RAM estimate	1,280 × assumed 20%	256 GB placeholder
RAM capacity per host at target	64 × 65%	41.6 GB

Reserve and overcommit

The N+1 result is a minimum host-failure reserve under the stated assumptions; maintenance may require another spare or capacity temporarily borrowed from a cloud provider. A two-host failure would leave 29 hosts in the 31-host design, supporting only 301.6 core-equivalents at the target, below the estimated 304 peak. The university should decide whether to fund N+2, allow temporary higher utilization or accept a controlled reduction in noncritical workloads. Define this trade-off by service priority before a failure occurs. Memory overcommit is more dangerous for latency-sensitive databases because ballooning or swapping can cause sharp response-time degradation.

Additional design considerations

The calculated minimum assumes a uniform workload and identical hosts. Real applications have different CPU profiles, and one large VM may not fit cleanly after a host failure even when aggregate core capacity appears sufficient. Use anti-affinity for replicated services, keep enough contiguous memory for large databases, and reserve storage and network

bandwidth for recovery. Recalculate after hardware selection because modern hosts may have a different core count and memory channel configuration than the existing fleet.

TASK 4 – Compare Virtual Machines and Containers (4 Marks)

Determine which workloads should run in virtual machines and which can run in containers.

Explain the architecture and characteristics of virtual machines and containers.

Compare VM and container isolation, operating-system requirements, startup time, resource overhead, portability and management.

Map at least five university workloads (for example, database, legacy student system, web portal, learning platform and analytics service) to VMs or containers.

Explain how Docker and a container orchestration platform such as Kubernetes can support deployment and scaling.

Identify situations where a VM is more appropriate than a container.

Evidence Expected: A VM-versus-container comparison and a workload mapping table supported by technical justification.

ANSWER

Comparison

Isolation	VM has separate guest kernel/virtual hardware; container shares host kernel.
OS	VMs support different guest OSs; containers require compatible kernel family.
Startup	VM typically seconds/minutes; container usually seconds or less.
Overhead	VM includes guest OS; container runtime/image layers are lighter.
Portability	VM depends on compatible virtual hardware; image runs on compatible container runtime.
Best use	VM for legacy/stateful/separate-kernel needs; container for stateless services and workers.

Workload placement

Legacy student system	VM: preserve supported OS/runtime.
SIS database	VM or managed DB: integrity, persistence, backup and licensing controls.
Web portal	Containers: stateless replicas behind load balancer.
LMS/API	Kubernetes containers: independently deploy and scale services.
Analytics jobs	Containers for reproducibility; VM/GPU node if special kernel/device is needed.
Examination service	VM or reserved container pool: predictable capacity during exam windows.

Docker and Kubernetes

Docker packages application/dependencies in images and runs isolated processes. Kubernetes schedules and reconciles replicas, services and deployments. Use resource requests/limits, probes, quotas and disruption budgets; horizontal autoscaling can adjust replica count from CPU, memory or custom metrics. VMs are preferable for different kernels, legacy appliances, vendor restrictions and stronger administrative boundaries.

Selecting workload boundaries

Use a VM when a workload requires its own kernel, legacy operating system, vendor appliance, isolated patch cycle or specific device support. Containers suit repeatable builds, rapid deployment and horizontal scaling. A product can use both: containerized web and API

components with a separate VM or managed service for its database. Persistent records must live in durable storage, not a container writable layer, so rescheduling a pod does not lose data. Build images from approved base versions, scan and sign them, keep secrets outside images and use private registries.

Kubernetes guardrails

Namespaces, service accounts, network policies and quotas help organize workloads, but college data isolation must also be enforced by application authorization and database design. Set CPU/memory requests for predictable scheduling and limits to prevent a runaway process. Readiness probes should stop traffic reaching an unready replica; liveness probes should be tuned to avoid restart loops under load. Use deployment rollouts with health checks and rollback. Containers share a host kernel, so privileged mode and host mounts can weaken isolation; limit these capabilities and keep worker nodes patched.

Stateful and stateless placement

A web front end is usually stateless if sessions are stored in a shared cache or token, allowing replicas to scale horizontally. A relational database is stateful and requires durable storage, backup, transaction consistency and careful failover; it should not be treated as an interchangeable container replica. A legacy student system may depend on a particular operating system, database driver or vendor-supported configuration and is often safest in a VM during initial modernization. Analytics workers can be containers if inputs and outputs are externalized; hardware-specific research code may need a VM with controlled GPU passthrough. Each mapping should be re-evaluated after compatibility and performance testing.

Image lifecycle

The image pipeline should define a base-image owner, support period and patch cadence. Rebuild images regularly instead of patching long-lived containers manually. Record image digests so the same artifact can be promoted from test to production. Scan for vulnerable packages and block deployment if critical policy thresholds are exceeded. Rollouts should use a canary or small percentage first, monitor errors and latency, then expand. Store deployment manifests in version control and keep a rollback version. These controls support reproducibility and accountability across university IT teams and affiliated colleges.

Operational security for containers

Use non-root containers, drop unneeded Linux capabilities, restrict hostPath mounts, apply seccomp/AppArmor profiles where available and avoid privileged mode. Separate workloads into namespaces and apply network policies. Keep cluster control-plane credentials out of application containers. Sign and scan images, patch worker nodes, and define who may access the cluster API. Monitor for unexpected process execution and outbound connections.

These protections reduce risk but do not make a shared-kernel container boundary equivalent to a VM boundary; sensitive or untrusted workloads may warrant dedicated VMs or nodes.

VM portability and lifecycle

A VM template should define virtual hardware, guest OS, monitoring, backup policy and network attachment. Thin VM templates simplify deployment but must be patched and versioned. Clones need unique machine identity and credentials. A VM moved to a host with different CPU features may encounter compatibility issues unless the cluster uses a compatible CPU baseline. Live migration also requires network and storage prerequisites. These operational details should be included in the comparison rather than describing portability as automatic.

Additional design considerations

Use deployment standards so that VM and container decisions remain reviewable. Each workload should have an owner, support tier, image or template version, network policy, data classification, backup method and retirement date. Containerization may be a step toward modernization, but it adds an orchestration control plane, registries, admission rules and cluster upgrades. Staff should practice restoring the service from source code, manifests and persistent data so that portability is proven rather than assumed.

TASK 5 – Design Virtualized Storage and Network Infrastructure (4 Marks)

Design storage and network resources that support the proposed virtualized data centre.

Explain storage virtualization and compare direct-attached storage (DAS), network-attached storage (NAS) and storage area networks (SAN).

Explain software-defined storage (SDS), thin provisioning, snapshots, replication and backup.

Explain network virtualization, virtual switches, VLANs, virtual networks and software-defined networking (SDN).

Propose separate logical network segments for administration, student services, databases and management traffic.

Recommend suitable storage and network approaches for performance, security and availability.

Evidence Expected: A labelled storage and network design with explanations of the selected technologies and their roles.

ANSWER

Storage and network design

DAS attaches disks to a host and limits sharing/mobility. NAS exposes files; SAN exposes block storage over a dedicated fabric, suitable for shared VM datastores when latency and support are adequate. Use redundant SAN or distributed SDS with SSD/NVMe tiers,

replication/erasure coding and capacity alerts. Thin provisioning allocates on use, so alert before pool exhaustion. Snapshots are short-term rollback, not backup; encrypt and replicate critical data to another failure domain and keep immutable/offline backups.

Segments

Administration	**VPN/bastion and MFA only; no student ingress.**
Student services / DMZ	WAF/load balancer to web/API only; rate limits.
Application	Permit only required service-to-service flows.
Database	Private; application identities only; encrypted connections.
Management/storage	Dedicated isolated network, admin-only, no public route.

Networking

Virtual switches connect virtual NICs; VLANs segment L2; virtual overlays extend logical networks; SDN applies centrally managed policies. Use redundant switches/links, default-deny firewall rules, micro-segmentation and encrypted management traffic.

Storage choices

SAN or well-supported SDS is suitable for shared VM disks when latency, multipathing and redundancy meet workload needs. NAS is effective for shared documents and user uploads; object storage can hold media, archives and immutable backups. DAS remains useful for disposable scratch work but should not be the only copy of critical records. Assign predictable IOPS tiers to SIS and examination databases, and lower-cost tiers to archival research data. Replication can copy corruption as well as valid changes, so combine it with point-in-time immutable backups and restore testing. Snapshots are short-term rollback points, can consume capacity as blocks change, and may be inconsistent unless coordinated with the application.

Network controls

Separate public ingress, student services, application, database, administration, hypervisor management and storage traffic. Allow only the load balancer to reach public application ports and only approved service identities to reach database ports. Management interfaces should be accessible through a secured bastion with MFA. VLANs provide basic layer-2 segmentation; SDN and distributed firewall rules apply policy to workloads even when they move between hosts. Redundant switches, links and routing paths reduce single points of failure. Back up switch, firewall and SDN configurations and test restoration.

Service classes

Classify storage into tiers based on recovery and performance needs. Tier 1 might host examination transactions and core SIS databases with low-latency redundant storage and frequent log protection. Tier 2 might hold LMS course files and user uploads with elastic capacity and replication. Tier 3 might contain research archives or backups where restore time is longer but retention is important. Document the acceptable latency, durability and

retention for each class. Capacity forecasting should include growth plus snapshot, replication and backup overhead; administrators should not promise all raw capacity to VMs.

Network implementation

Start with a documented address and VLAN plan, including gateways, DNS, DHCP, routing, firewall owners and change control. Avoid using a flat network where management interfaces share the student-facing segment. Control east-west traffic between application components as carefully as internet ingress. Use TLS for user and service connections, and protect management protocols. For partner colleges, apply per-tenant identity and policy at the application layer while using network segmentation as an additional boundary. Test the design by verifying allowed flows and confirming that prohibited paths fail and are logged.

Capacity and durability arithmetic

If a 40-TB raw disk pool uses two-way replication, usable capacity is at most about 20 TB before filesystem and snapshot overhead; three-way replication reduces it further. This is only an illustration because the source servers have separate local disks, not one shared pool. A realistic target must include the data that is actually migrated, growth, spare capacity, backup retention and recovery copies. Thin provisioning does not create physical capacity: monitor both logical allocation and actual consumption. Replication can improve availability but should not be counted as a separate backup if it mirrors accidental deletion or ransomware.

Segmentation validation

Create an explicit allowlist of required flows. For example, internet clients may reach only the WAF; the WAF may reach approved web services; web services may reach designated API endpoints; application identities may connect to database ports; administrators may reach management services only through a bastion. Test blocked flows from a student VLAN and from one college tenant to another. Store rule changes in version control, require review for broad rules, and remove temporary exceptions after an event.

Additional design considerations

Record the storage failure domain and recovery behavior for each tier. A replica in the same rack protects against a disk failure but not against rack power loss; a replica in another site may improve disaster recovery but introduce latency and transfer cost. Define retention by data type and legal or institutional need. Test that administrators can restore a single file, a VM disk and a database to a selected recovery point without overwriting the production copy.

TASK 6 – Plan Resource Allocation, Scheduling and Monitoring (5 Marks)

Develop a resource-management strategy that allocates infrastructure capacity efficiently.

Explain CPU, memory, storage and network resource allocation to VMs and containers.

Compare static allocation and dynamic allocation, including resource reservations, limits and shares.

Explain load balancing, workload scheduling, auto-scaling and high availability.

Identify suitable performance indicators: CPU utilization, memory utilization, disk I/O, network throughput, latency and VM availability.

Propose monitoring and alerting methods, and explain how the administrator should respond to resource contention.

Evidence Expected: A resource-allocation policy table and a monitoring plan that includes measurable indicators, thresholds and corrective actions.

ANSWER

Policy

CPU	Reservations for SIS/exams; shares under contention; cap batch workloads.
Memory	Right-size from telemetry; reserve DB/cache memory; avoid chronic ballooning/swapping.
Storage	Thin provision with 70/85% alerts; IOPS QoS for databases; separate backup pool.
Network	QoS for exam/SIS; cap bulk jobs; size links from measured throughput.
Containers	Requests/limits, namespace quotas, affinity, probes and disruption budgets.

Monitoring

Centralize hypervisor, VM, container, storage and network metrics/logs. Initial alerts: host CPU >80% for 5 min; memory pressure >85%; datastore latency >20 ms; link >80%; service p95 SLO breach or failed health check. Tune using a 2–4 week baseline. Respond by checking changes and affected service, locating hot hosts/storage, rebalancing or live migrating, scaling stateless replicas, capping runaway jobs, prioritizing student/exam services, and tracking root cause.

Allocation semantics

Reservations provide guaranteed minimum capacity and should be used for critical SIS and examination systems after aggregate capacity is checked. Limits cap consumption but can throttle service if set too low. Shares express priority only during contention; they do not reserve resources. Static allocation is predictable but can strand capacity, while dynamic allocation responds to demand but requires accurate telemetry and guardrails. Start with measured baselines and adjust through controlled tests. Autoscaling should consider queues, request rates and latency as well as CPU: a database bottleneck may not improve by adding web replicas. Pre-scale before scheduled examinations because new instances take time to become healthy.

Monitoring response

Alerts need a named owner, severity, dashboard, first checks and escalation path. A CPU alert can reflect real demand, an inefficient query, a recent deployment or a noisy neighbor; adding hosts without investigation can hide a defect and raise cost. For memory pressure, inspect working sets and swap. For storage latency, find whether the array, path or one

workload is responsible. After service recovery, record impact, timeline, cause and preventive action. This turns monitoring into an operational feedback loop rather than a collection of charts.

Resource priority example

During result publication, reserve capacity for database writes, identity services and the result portal. A research batch workload may receive lower shares and a limit so it cannot monopolize storage or CPU. Outside the event, unused reserved capacity can be reclaimed if the platform supports safe overcommit, but do not remove the minimum failover reserve. Set autoscaling thresholds with a stabilization period to prevent repeated scale-up/scale-down cycles. Enforce scale-out ceilings to control cost and protect databases from excessive connection load. The service owner should define the user-visible action if the ceiling is reached, such as a queue or maintenance message.

Capacity review cadence

Review utilization weekly during a semester and daily during major enrollment or examination events. Compare forecast with actual host use, per-service latency, storage growth and cloud spend. Investigate persistent high CPU or memory pressure before adding more VMs, since inefficient queries and memory leaks can imitate lack of capacity. Review low-use VMs and reclaim stale allocations after owner approval. Preserve data and audit retention when deleting resources. A quarterly capacity report should state the peak observed, forecast growth, failure reserve and procurement lead time.

Indicator interpretation

CPU utilization alone can hide scheduling contention; a host may report moderate usage while VMs wait for CPU time. Memory utilization should be interpreted with active memory and reclaim behavior. Storage latency needs read/write breakdown and queue depth because a small number of large I/O jobs can affect all VMs. Network throughput should be paired with packet loss, retransmits and interface errors. VM availability is a service-level measure; a powered-on VM can still serve failed requests. Pair infrastructure metrics with synthetic transactions and user-facing response time so alerts reflect actual impact.

Response thresholds as policy

An 80% CPU alert is a starting point, not a universal threshold. A brief burst may be harmless, while a lower sustained level with high CPU-ready time may harm latency. Establish warning and critical thresholds, persistence duration, owner and action. During a major event, review dashboards more frequently and use a change freeze for nonessential production releases. After an alert, retain logs and metric snapshots before scaling, since evidence can identify a regression. Record whether the action resolved user impact and whether it created excess capacity or cost.

Additional design considerations

Operational reports should distinguish consumed capacity from configured capacity. Over-provisioned CPU or memory can make a cluster appear full even while workloads use little, whereas a small but latency-sensitive workload can be unhealthy despite low aggregate utilization. Review trends by application, host, tenant and failure domain. Maintain a forecast of the date when headroom will fall below the procurement lead time so the university can act before resource pressure reaches users.

TASK 7 – Evaluate Data Centre Availability, Security and Disaster Recovery (4 Marks)

Identify the risks associated with a virtualized data centre and propose suitable controls.

Explain hypervisor security, VM isolation, secure VM images, patch management and privileged access control.

Discuss VM escape, VM-to-VM attacks, insecure snapshots, VM sprawl and denial-of-service risks.

Recommend network segmentation, encryption, multi-factor authentication, role-based access control and audit logging.

Explain backup, replication, snapshots, live migration and disaster recovery.

Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO), and propose suitable targets for at least two university applications.

Evidence Expected: A risk-and-mitigation table and a disaster-recovery plan with justified RTO and RPO values.

ANSWER

Risk controls

Hypervisor compromise/VM escape	**Minimize interfaces; harden and patch; MFA/RBAC; isolate management.**
VM-to-VM attack	Default-deny segmentation, workload identities, host firewall and flow logs.
Insecure snapshots/images	Encrypt, restrict and expire snapshots; sanitize templates/secrets.
VM sprawl	Owner, purpose, expiry, quotas and monthly inventory.
Denial of service/noisy neighbor	Resource caps/reservations, quotas, rate limits and capacity monitoring.
Admin credential theft	MFA, bastion, time-bound roles and immutable audit logs.

Recovery objectives

RTO is maximum acceptable recovery time; RPO is maximum acceptable data loss interval. SIS: RTO 2 hours/RPO 15 minutes using frequent log backups and replication. LMS: RTO 4 hours/RPO 1 hour. Examination results during publication: RTO 1 hour/RPO 5 minutes. These require funded secondary capacity, application-consistent backups, encrypted replication, immutable daily backups, runbooks and quarterly restore/failover tests. Live

migration supports planned maintenance within a healthy site; it does not replace disaster recovery.

Risk scenarios

VM escape targets the host or hypervisor, so minimize installed components, patch supported versions, restrict device passthrough and isolate administration. VM-to-VM attacks remain possible across shared hosts; apply default-deny network rules. Snapshots and images can contain credentials or personal data, so encrypt and restrict them and expire temporary copies. VM sprawl leads to unpatched, unmonitored systems; require owner, purpose, expiry and backup policy for each workload. Denial of service can be intentional or accidental; quotas, rate limits, reservations and alerts reduce impact. For each risk, name a control owner and detection signal.

Recovery operations

A backup plan includes databases, configuration, identity dependencies, certificates, encryption keys and runbooks. Replication reduces recovery delay but should target a separate failure domain. Restore order should cover identity, DNS, network/security services, databases, SIS/LMS and then public portals. Test a sample restore monthly and a broad failover before a major examination period. Record achieved RTO/RPO and inconsistencies. If tests miss the target, improve capacity and process or revise the objective openly; an untested target is only an assumption.

Security baseline

Maintain a hardened host baseline: restrict management interfaces, disable unused services, use trusted time and logging, apply security updates, protect configuration backups and review vendor advisories. Secure VM images with current patches, endpoint protection where appropriate, limited local accounts and removal of test credentials. Encrypt backups and replication channels, and separate backup administration from production infrastructure accounts. Test restore credentials independently. Use role-based access so an application developer can manage their own deployment but cannot alter the hypervisor or inspect other tenants.

Recovery prioritization example

After a major site failure, restore identity and DNS dependencies first, then database services, SIS and examination systems, LMS, and lower-priority public or research services according to the agreed business impact analysis. The SIS target of two hours and the examination target of one hour require pre-positioned recovery capacity, tested data replication and assigned operators. If only backups are available, recovery may take much longer than the target because data must be restored and validated. Record recovery dependencies, contact

lists and decision authority in the runbook, and ensure a copy is available offline or outside the primary site.

Defense in depth

The hypervisor, management console, storage, guest operating systems, applications and identity provider all form part of the security boundary. Apply standard operating-system controls inside each guest; virtualization does not remove endpoint patching, malware protection or application authorization. Protect management APIs with TLS, MFA and source-network restrictions. Back up configuration and verify that a recovery administrator can reach it without the primary identity service. Audit snapshot export and VM console access because these channels can expose data. Review vendor security advisories and use a documented patch window that preserves cluster capacity during rolling updates.

Recovery decision points

For a site outage, the incident lead should confirm the failure scope, declare disaster recovery, identify the latest consistent data point and start services in dependency order. Business owners decide whether to operate in a degraded mode, such as delaying noncritical analytics while student services recover. After failover, prevent the original site from rejoining as an active writer until data reconciliation is completed. Failback should be a planned change with verified replication direction and a second rollback plan. Record the actual recovery point and time; these determine whether the stated objectives are realistic.

Additional design considerations

Security controls should be checked during normal operations and after platform changes. Review privileged group membership, stale VM images, expired snapshots, open firewall rules, patch compliance and backup immutability. An incident response plan should include isolation of a compromised VM, preservation of evidence, credential rotation and assessment of whether snapshots or replicas contain the same compromise. Recovery tests should verify security settings as well as service availability.

TASK 8 – Create the Virtualized Cloud Data Centre Architecture (5 Marks)

Construct an end-to-end architecture integrating compute, storage, network, virtualization and management components.

Draw a labelled architecture showing users, internet or campus network, firewall, load balancer, physical hosts, hypervisor layer, VMs/containers, virtual network, storage and monitoring.

Show how the university applications are distributed across hosts and how shared resources are allocated.

Include a management plane for provisioning, monitoring, logging and policy enforcement.

Show backup or replication to a secondary site or cloud region.

Justify the architecture in terms of scalability, performance, fault tolerance, security and cost.

Evidence Expected: A complete, clearly labelled architecture diagram and a written justification for each major component and design decision.

ANSWER

Architecture

Campus/Internet and college users → DDoS protection/WAF → redundant firewalls → load balancer/API gateway → DMZ web → private application tier (legacy/stateful VMs and stateless Kubernetes LMS/API/web) → isolated database tier. Type-1 hypervisor cluster with N+1 capacity; redundant switches and virtual network; shared SDS/SAN datastore; object storage for media/backups. Management plane: identity/MFA, provisioning, image registry, patching, monitoring, logs and policies. Replicate encrypted data/backups to a second site/cloud region.

Justification

Segmentation limits lateral movement. HA restarts workloads after host failure; Kubernetes reschedules stateless pods; live migration supports maintenance. Redundant power/cooling, switches, links and storage address component failure; second-site recovery addresses site failure. Begin hybrid: campus for controlled workloads and measured cloud burst/DR to manage cost.

Component roles

A WAF filters web attacks; the load balancer checks health and distributes requests; an API gateway authenticates clients, applies quotas and routes versions. Applications should access data using workload identities and narrow permissions. The database tier has no public route. Separate management-plane connectivity prevents public workloads from reaching hypervisor and storage controls. Monitoring collects metrics and logs from each layer. Place redundant replicas across racks or zones using anti-affinity so one failure cannot remove every copy. State is handled by database/storage services designed for consistency and recovery.

Failure handling

Live migration supports planned maintenance when shared storage, compatible CPUs and network continuity are available; it does not protect against a site outage. High availability can restart VMs after a host failure, while Kubernetes can reschedule stateless pods. Database failover needs quorum and split-brain protection. Define service-level health checks, scale-up times and reserved quotas before peak periods. Conduct exercises for host loss, network partition, certificate expiry and failed backup restoration. A second site should be operable even if the primary management plane is unavailable.

Architecture deployment sequence

First deploy redundant network, identity, management and monitoring foundations. Next establish the hypervisor cluster and shared storage, then build approved VM images and segregated virtual networks. Deploy low-risk web workloads and test provisioning, migration and alerts. Add database and stateful services only after backup and restore are proven. Finally, connect the secondary site and run a controlled failover test. This sequence avoids placing critical applications on a platform whose management, logging or recovery services are not yet ready.

Failure-domain review

A cluster can appear redundant while retaining a common failure point such as one power feed, top-of-rack switch, storage controller, DNS service or management appliance. Map physical and logical dependencies and confirm that redundant components are genuinely independent. A single storage system with two controllers still has a site-level dependency. The secondary location needs network routes, identity integration, capacity, data protection and operator access. Keep configuration as code or securely backed up so the recovery environment can be rebuilt without relying on a failed production control plane.

Control plane and data plane

The control plane includes hypervisor management, orchestration, identity, policy, monitoring and provisioning. The data plane carries student/API requests, database traffic and storage I/O. A management-plane outage may prevent administrators from creating or moving workloads even when existing VMs continue to run. Protect and back up this plane separately, and document manual recovery steps. Keep management APIs off public networks and restrict access to approved administrator devices. Monitoring should cover the control plane itself, including cluster-manager health, certificate expiration and configuration backup status.

Scale-out boundaries

Stateless web services often scale horizontally, while relational databases may be limited by write throughput, connection count or lock contention. A replicated cache can reduce database pressure but introduces invalidation and consistency questions. Video delivery can dominate network egress even when CPU remains low; a CDN or object-storage delivery path may be more efficient. Identify bottlenecks in each tier before selecting an autoscaling target. Load testing should include realistic traffic mix and failure cases, not merely a high number of simple requests.

Additional design considerations

The architecture should use independent failure domains for components whose loss would interrupt many services. Consider dual power feeds, redundant network paths, separate storage controllers and multiple control-plane instances. Avoid relying on a single

management appliance or a single administrator credential. Draw arrows for both user traffic and management/backup traffic so reviewers can see which paths are allowed. The diagram should be accompanied by the ownership and recovery behavior of each major component.

TASK 9 – Hands-on Implementation and Validation (5 Marks)

Build a small proof-of-concept that demonstrates virtualization and resource management.

Create at least one VM using VirtualBox, VMware, Hyper-V or KVM, or create a container using Docker.

Deploy a simple university-related service, such as a web page, student-record API or database-backed application.

Configure CPU, memory and storage resources and record the selected values.

Perform functional tests and at least one resource or load test. Record the observations.

Capture screenshots of the VM/container configuration, running service, resource-monitoring output and test results.

Document installation steps, commands, configuration details, issues faced and how they were resolved.

Evidence Expected: Reproducible implementation steps, source/configuration files, annotated screenshots and actual test evidence. Do not present simulated results as actual execution.

ANSWER

Proof of concept

No VM/container was actually run for this document, so no execution outcome or screenshot is claimed. On a lab computer, create a small Flask API with /health returning status ok and /students/<id> returning a sample record for a known ID or 404 otherwise. Pin Flask in requirements.txt. Dockerfile: FROM python:3.12-slim; WORKDIR /app; COPY requirements.txt .; RUN pip install --no-cache-dir -r requirements.txt; COPY app.py .; EXPOSE 8080; CMD ["python","app.py"]. Bind to 0.0.0.0:8080.

Commands to run and document

docker build -t university-api:1.0 .

docker run -d --name university-api --cpus=1 --memory=512m -p 8080:8080 university-api:1.0

curl -i http://localhost:8080/health

curl -i http://localhost:8080/students/1001

curl -i http://localhost:8080/students/unknown

docker stats --no-stream university-api

Capture actual outputs, versions, host OS, measured CPU/memory, errors and screenshots. This is a reproducible procedure, not proof of execution.

Evidence checklist

Add authentic screenshots of container config, running API, resource monitoring and successful/failure requests. Record test date and issues/resolution; do not label expected output as observed output.

Testing evidence

Run functional tests for health, a valid student record, an unknown ID returning 404, malformed input and unauthorized access. Record image tag, dependency versions, host OS, Docker version and resource limits. For a controlled load test, gradually raise concurrency while measuring request rate, error rate, p95 latency, CPU and memory; stop before affecting a shared system. The report must state test duration and request mix. Attach real screenshots of configuration, service responses and resource monitoring. If Docker is unavailable locally, use an approved lab. Do not replace actual evidence with expected output; explain errors and their resolution.

Evidence record template

For every test, record the date and time, tester, host model, operating system, runtime versions, image tag, CPU/memory configuration, command, expected behavior, actual output and pass/fail result. For load tests, include the client tool/version, request distribution, concurrency ramp, duration and any rate limits. Capture screenshots directly from the terminal or dashboard and retain raw output when possible. Use synthetic student records rather than real personal data. A concise observation should state whether the service met its expected behavior and what the resource graph showed; do not infer production readiness from a small local test.

Limitations

A single Docker container on a laptop demonstrates packaging and basic resource limits; it does not prove high availability, multi-node scheduling, security isolation or production scaling. A more complete extension is to deploy the same API as multiple Kubernetes replicas, terminate one pod and observe rescheduling, then perform a controlled load test. Any such additional exercise must be carried out and documented by the student. Clearly distinguish lab evidence from production validation.

Example result interpretation form

After running the tests, record observations in a table: test name, expected result, actual HTTP status/body, CPU and memory, duration and conclusion. For example, the unknown student ID should return 404 without exposing stack traces; an oversized request should be rejected; the health endpoint should not disclose internal configuration. If the container reaches its memory limit, record whether it is killed and how the restart policy behaves. Any result must come from the observed environment. If the API fails, preserve the error log, explain the correction and repeat the test rather than reporting only the expected success.

Reproducibility

Pin dependency versions and specify the exact build context so another student can reproduce the image. Avoid the latest tag in a final test because it may resolve to different software later. Record whether the test ran on Windows with Docker Desktop, Linux Docker Engine or a university cluster. Document required permissions and ports. Keep the test service isolated and use synthetic data. A reproducible implementation is valuable even when the small proof-of-concept does not demonstrate enterprise availability; it shows which claims were validated and which remain design proposals.

Additional design considerations

A lab proof-of-concept validates a narrow slice of the proposal: image packaging, process isolation, endpoint behavior and resource limits. It does not validate production capacity, tenant security, durable backups, high availability or the 99.9% target. State this boundary clearly in the report. If additional tests are performed, preserve their raw logs and identify the exact environment so another reviewer can distinguish repeatable evidence from a design recommendation.

TASK 10 – Evaluate Cost, Performance and Final Recommendation (3 Marks)

Evaluate the proposed solution against the existing physical data-centre arrangement.

Compare the current physical-server model and proposed virtualized model in terms of hardware cost, electricity, cooling, maintenance, licensing and administration effort.

Prepare a simple three-year cost comparison using clearly stated assumptions. Separate one-time capital expenditure from recurring operational expenditure.

Evaluate expected improvements in utilization, scalability, availability and manageability.

Identify risks such as licensing costs, vendor lock-in, skills requirements, hardware failure and migration downtime.

Provide a phased migration roadmap, identifying which applications should be migrated first and why.

Evidence Expected: A three-year cost table, risk analysis, migration roadmap and evidence-based final recommendation.

ANSWER

Cost assumptions

Illustrative ₹ lakh, not quotes: physical replacement servers 20×₹5L; 600 W/server; power ₹8/kWh; PUE 1.8; maintenance ₹1.5L/server/year; ops ₹12L/year. Virtual model: 8 hosts×₹12L + ₹20L shared storage/network; 800 W/host; PUE 1.5; maintenance ₹2L/host/year; ops ₹18L/year. Licensing excluded pending vendor quotes.

Three-year comparison

Cost (₹ lakh)	Physical 20-server	Virtual 8-host average design
Hardware/shared storage (year 0)	100	116
Power + cooling (3 years)	45.4	25.2
Maintenance (3 years)	90	48
Operations (3 years)	36	54
Licensing / software	Not included	Not included
Illustrative total	271.4	243.2

Working and recommendation

Physical power/year = 20×0.6 kW×8,760×₹8×PUE 1.8 = ₹15.15L; 3 years ≈₹45.4L. Virtual = 8×0.8×8,760×₹8×1.5 = ₹8.41L/year; 3 years ≈₹25.2L. The 31-host peak-ready cluster costs more than this average-load case. Include current license quotes, tax, cloud egress, skills and measured peak hours in final TCO. Risks: migration downtime, lock-in, skills gap, host/storage failure and core-based license increases. Roadmap: inventory/baseline; build and test cluster/network/backup; pilot low-risk website/dev; migrate library/web; migrate SIS/LMS with owner validation and rollback; migrate examination service in maintenance window; test DR; retire old hardware after acceptance.

Cost interpretation

The three-year estimate separates capital hardware from recurring power, maintenance and labor, but remains illustrative. Existing equipment may be sunk cost; replacement value matters only if refresh is due. Virtualization may require hypervisor subscriptions, backup software, management tooling, database licensing and staff training. Power tariffs and PUE vary by facility. Include written quotes and model low, expected and peak demand. Compare average-load private capacity, peak-ready private capacity and hybrid burst. A hybrid option may be economical for short peaks but expensive if utilization stays high or data egress is large; calculate a break-even number of peak hours.

Migration governance

Give every migration a service owner, change window, backup and rollback trigger. Validate DNS, certificates, integrations and transaction counts after cutover. Prevent two active writable copies. Keep the former server isolated but available for a defined rollback period. Track performance, incidents and costs through each wave. Securely erase retired media, revoke credentials and update asset records only after acceptance. Risk controls include phased migration, tested restores, staff training and license review.

Comparison limitations

The illustrative model assumes fixed power draws and support costs. Actual servers draw different power at idle and under load, and shared storage/network gear also consumes electricity. Cooling efficiency changes with utilization and season. Maintenance costs may include warranties, spare parts, on-site response and software support. Operational labor

estimates should represent actual staffing rather than assume consolidation removes all administration. A cloud comparison must include compute, storage, snapshots, managed services, data transfer, support and minimum commitments. Use a consistent discount rate and tax treatment if required by the institution.

Decision rule

The university should approve the investment only if the revised three-year total cost fits budget and the selected option meets performance, recovery and security requirements. If costs are close, favor the design that meets service targets and can be operated with available skills. Use a pilot to validate the estimated consolidation ratio and power savings. Retain a contingency for licensing changes and workload growth. Report best-case, expected and conservative scenarios so the decision is not dependent on one optimistic estimate.

Lifecycle view

A physical server model has replacement cycles, warranties, spare inventory and a large number of independent operating-system installations. Virtualization reduces host count but concentrates risk in fewer, more powerful machines and shared storage. It can lower power and simplify maintenance, yet the cluster requires skilled administrators and may create subscription costs. Cloud capacity avoids some upfront hardware costs but introduces recurring consumption and data-transfer charges. These options should be evaluated over a full lifecycle, including refresh and exit costs. A three-year model should also state residual value or explain why it is excluded.

Performance validation

Compare before and after using the same transaction mix and service-level indicators. Check that consolidation has not increased database latency or contention. Monitor CPU ready time, memory pressure, storage latency, network drops, response-time percentiles and error rates. Run tests during both average and peak conditions. If a workload performs worse after migration, investigate virtual device drivers, NUMA placement, storage pathing and overcommit before adding capacity. Maintain a rollback path until the application owner accepts results.

Additional design considerations

Financial review should include uncertainty. Present the cost range when server prices, energy use or cloud demand are not yet known, and identify the assumption that has the largest effect on the total. Include staff time for migration, training and security review, plus the cost of running old and new platforms in parallel during cutover. Revisit the business case after one full academic year using measured power, support tickets, capacity use and cloud invoices.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Virtualized Infrastructure (8 Marks)

Changed Requirement

The university introduces a digital learning platform and extends selected services to 10 affiliated colleges. The expected peak workload increases to five times the current peak. The system must achieve 99.9% availability, provide secure isolation between colleges and remain within a limited operating budget.

Re-evaluate your original design. You may RETAIN, MODIFY or REDESIGN the proposed infrastructure.

Impact Analysis – 2 Marks

Identify changes in workload, users, storage, network traffic and availability requirements.

Explain how multi-tenant access affects VM/container isolation, network segmentation and access control.

Identify which original capacity and cost assumptions are no longer valid.

Capacity and Availability Evaluation – 2 Marks

Estimate the additional CPU, memory, storage and network capacity needed for the five-times peak workload.

Evaluate whether the proposed number of hosts and the existing failover arrangement are sufficient.

Explain the meaning of 99.9% availability (approximately 8.76 hours of downtime per year) and identify measures needed to approach it.

Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the original infrastructure and justify the decision.

Describe the revised architecture. Consider clustering, live migration, auto-scaling, container orchestration, multi-zone deployment or hybrid-cloud capacity where appropriate.

Explain changes to resource-allocation policies, tenant isolation, backup and disaster recovery.

Evidence-Based Defence – 2 Marks

Support the revised design with capacity calculations and clearly stated assumptions.

Compare the cost and operational complexity of the original and revised designs.

Discuss trade-offs involving performance, availability, security, licensing and management effort.

Evidence Expected: A technically justified decision supported by revised architecture, capacity calculations, availability considerations, security controls and cost comparison.

ANSWER

Impact analysis

Stated case: peak becomes 5×304=1,520 core-equivalents, new learning platform and 10 colleges. It adds API, storage/media and network traffic; tenant count increases identity, authorization, data segregation, quotas, logging and support needs. The 8-host average design, single site and periodic backup are insufficient. The duplicated rubric's 10×/40-college case is considered below as sensitivity.

Capacity and availability

At 16 cores/host and 65%, usable=10.4 core-equivalents. Fivefold peak: ceil(1,520/10.4)=147 active hosts; N+1=148. Tenfold sensitivity: ceil(3,040/10.4)=293; N+1=294. This conservative direct scaling must be validated because server peak concurrency may differ; memory, IOPS, network, GPU and licensing may bind first. Benchmark representative concurrent transactions and media egress, keep 30% headroom and N+1 per failure domain. 99.9% allows 0.001×365×24=8.76 hours/year. A single site cannot cover site/power/network failure; use independent zones/sites, redundant edge/network/storage, health-checked routing, automated failover and regular recovery drills.

Decision and redesign

Modify to hybrid multi-zone. Retain a Type-1 private cluster for SIS, database and legacy workloads; run stateless LMS/API/web on Kubernetes across at least three failure zones, using autoscaling from CPU and request/queue metrics. Add API gateway/WAF, tenant-aware identity and authorization, quotas, network policies, encrypted secrets/data, and per-tenant audit. Use a second region/site, asynchronous replication where appropriate, immutable backups and tested failover. Autoscaling requires capacity/quota and cannot conjure capacity instantly.

Cost and trade-offs

Use IaaS for controlled legacy VMs, PaaS/managed Kubernetes and database services when controls meet university needs, and SaaS where the service boundary is acceptable; consider DRaaS/object storage after recovery-cost tests. A 148-host on-premises peak build may be capital-heavy at low off-peak utilization. Cloud burst converts some peak to variable cost, but sustained 10× use and egress can be expensive. Recalculate 3-year TCO with measured peak hours and provider/license quotes. Redundancy and isolation improve resilience but add cost/complexity; managed services reduce operations while increasing provider dependency. Adopt modified architecture after representative load test and successful DR exercise.

Tenant isolation

Each college needs distinct identity groups, roles, quotas and audit access. Enforce authorization in every API request, not only in the interface. Validate tenant identifiers on the

server and use safe database queries. Depending on risk, separate schemas, encryption keys or databases. Apply network policies so one college cannot reach another tenant's services; prevent tenant administrators from viewing platform secrets or other tenants' logs. Run cross-tenant access tests after releases and retain protected audit logs.

Service level and availability

Define the 99.9% measurement boundary, planned maintenance treatment and interval. Its annual error budget is 8.76 hours; reserve it for incidents rather than routine work. Redundant DNS, edge connectivity, firewalls, storage paths and power should span independent failure domains. Use useful health checks, automated failover for stateless services and carefully controlled database promotion to prevent split brain. During examination periods, assign an incident commander and communications plan. Game days should test host loss, zone outage, certificate expiry and backup recovery; record detection, decision and restoration times.

Decision and validation

The fivefold arithmetic assumes all present peak CPU demand grows linearly and at once. Validate this conservative bound with per-application CPU, memory, database connections, IOPS and egress measurements during an LMS pilot. Load-test login, course access, video retrieval, submissions and results publication. Database and identity services may scale differently from stateless web replicas. Modify the design to retain private virtualization for legacy/stateful systems and use multi-zone Kubernetes or managed PaaS for elastic LMS/API services, plus a secondary recovery site. Revisit capacity after the first semester and size from observed use. Proceed only after load, security, restore and failover tests pass.

Tenant onboarding and offboarding

Define a repeatable onboarding process for each college: verify the organization, establish identity federation or accounts, assign roles, allocate quotas, agree on data retention and configure tenant-specific API access. Test that a user can access only their own records and services. On offboarding, revoke credentials and tokens, export or delete data according to agreement, preserve legally required audit records and remove unused infrastructure. Document support escalation and incident notification contacts for each tenant. This lifecycle prevents tenant separation from being treated as a one-time network configuration.

Sensitivity case and recommendation

For the 10×/40-college rubric case, peak CPU demand is estimated at 3,040 core-equivalents and 294 hosts including one N+1 spare under the same conservative assumption. This is not an immediate purchase recommendation; it is evidence that the original capacity assumptions cannot simply be reused. The university should validate whether all services scale together, use managed services for elastic demand where appropriate, and place a procurement or cloud quota trigger before expected saturation. Review actual demand after enrollment and

each academic term, and expand only after the workload evidence supports the next investment.

Multi-tenant service governance

Affiliated colleges may have different user populations, course calendars, data retention needs and administrator roles. Define a service catalog and shared responsibility matrix: central IT secures the platform and shared identity; each college manages authorized users and local content; application owners maintain data-level controls. Establish incident reporting, maintenance notice, quota requests and service support escalation. Audit tenant provisioning and periodic access review. Ensure operational dashboards aggregate platform health without revealing another college's private records to tenant administrators.

Availability economics

To approach 99.9%, budget for redundant capacity and failure testing. If 8.76 annual hours are consumed by one long maintenance or site event, there is little error budget left for smaller incidents. Multi-zone deployment, redundant network paths and automated health checks reduce common failures but do not eliminate software bugs, identity outages or operator mistakes. Use staged deployments, canary releases, tested rollback and immutable backups. A secondary region provides protection against site-level events, but replication and data transfer add cost. The university should choose recovery strategy based on the business impact and measured recovery time, not on architecture fashion.

Additional design considerations

The revised service should include service-level objectives for each tier, not only one platform-wide availability number. Student login, learning content, assignment submission and examination results may have different recovery priorities. Measure them with synthetic transactions from campus and partner networks. Define a degraded mode when the full target cannot be met, such as read-only access to course material while submissions are queued. Publish responsibilities and incident contacts to all participating colleges before onboarding.

.Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Existing Data Centre and Requirements	L2 – Understand / Analyse	3
Task 2	Virtualization Fundamentals and Hypervisors	L2 – Understand	3
Task 3	Server Consolidation and Resource Calculations	L3 – Apply	3

Task	Title	Bloom's Level	Marks
Task 4	Virtual Machines and Containers	L4 – Analyse	5
Task 5	Virtualized Storage and Network Infrastructure	L3/L4 – Apply / Analyse	4
Task 6	Resource Allocation, Scheduling and Monitoring	L4 – Analyse	5
Task 7	Availability, Security and Disaster Recovery	L4 – Analyse	3
Task 8	Virtualized Cloud Data Centre Architecture	L6 – Create	5
Task 9	Hands-on Implementation and Validation	L3 – Apply	4
Task 10	Cost, Performance and Final Recommendation	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4/L5/L6 – Analyse / Evaluate / Create	8
	TOTAL		50